

EE2801 - DSP LAB

EXPERIMENT-3

Fixed-Point Division using Newton-Raphson iterations

EE24BTECH11012 - BHAVANISANKAR G S

February 21, 2026

Contents

1	Aim	2
2	Newton-Raphson Method for Reciprocation	2
2.1	Derivation of the Iteration Formula	2
2.2	Proof of Quadratic Convergence	3
2.3	The Mathematical Necessity of Normalization	3
3	Making an Initial Guess	4
3.1	Unconstrained Linear Minimax Approximation	4
3.2	Constrained Linear Approximation (Hardware Optimized)	4
3.3	Quadratic Minimax Approximation	5
3.4	Comparison of Methods	5
4	Algorithm Summary	5
5	Hardware Validation Test Cases	5
6	Results	6
7	Observations	9
8	Conclusion	9

1 Aim

To perform fixed point division by the method of Division by Reciprocation using Newton-Raphson method. The initial guess for Newton-Raphson method has been done in three ways -

- The optimal unconstrained linear minimax approximation.
- The hardware-constrained linear approximation (slope fixed to 2).
- The quadratic minimax approximation.

2 Newton-Raphson Method for Reciprocation

Given a continuously differentiable function $f(x)$ and a current approximation x_n near the true root, we can approximate the curve of the function by its tangent line at the coordinate $(x_n, f(x_n))$. The slope of the tangent line at this point is given by the derivative, $f'(x_n)$. Using the point-slope form of a linear equation, the equation for this tangent line is:

$$y - f(x_n) = f'(x_n)(x - x_n) \quad (1)$$

The algorithm assumes that the x -intercept of this tangent line provides a better approximation of the true root than x_n . To find this intercept, we set $y = 0$ and define the resulting x -coordinate as our next approximation, x_{n+1} :

$$\begin{aligned} 0 - f(x_n) &= f'(x_n)(x_{n+1} - x_n) \\ -f(x_n) &= f'(x_n)(x_{n+1} - x_n) \end{aligned}$$

Assuming the derivative is non-zero ($f'(x_n) \neq 0$), we divide both sides by $f'(x_n)$:

$$-\frac{f(x_n)}{f'(x_n)} = x_{n+1} - x_n \quad (2)$$

Rearranging the terms to solve for x_{n+1} yields the standard Newton-Raphson update rule:

$$\mathbf{x}_{n+1} = \mathbf{x}_n - \frac{\mathbf{f}(\mathbf{x}_n)}{\mathbf{f}'(\mathbf{x}_n)} \quad (3)$$

To perform division N/D in hardware, it is highly efficient to compute the reciprocal of the denominator ($1/D$) and multiply the result by the numerator N . The Newton-Raphson method is utilized to iteratively refine a guess for this reciprocal.

2.1 Derivation of the Iteration Formula

We seek the root of a function $f(x)$ such that $f(x) = 0$ when $x = 1/D$. A mathematically convenient choice that avoids hardware division is:

$$f(x) = \frac{1}{x} - D \quad (4)$$

The derivative of this function with respect to x is:

$$f'(x) = -\frac{1}{x^2} \quad (5)$$

The standard Newton-Raphson update rule is:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \quad (6)$$

Substituting our specific function and its derivative:

$$\begin{aligned} x_{n+1} &= x_n - \frac{\frac{1}{x_n} - D}{-\frac{1}{x_n^2}} \\ &= x_n - \left(\frac{1}{x_n} - D \right) (-x_n^2) \\ &= x_n + x_n - D \cdot x_n^2 \end{aligned}$$

Factoring out x_n , we arrive at the final, hardware-friendly iteration formula:

$$\mathbf{x}_{n+1} = \mathbf{x}_n(2 - \mathbf{D} \cdot \mathbf{x}_n) \quad (7)$$

This formula is exceptionally well-suited for DSP and FPGA implementation because it exclusively requires multiplication and subtraction.

2.2 Proof of Quadratic Convergence

The Newton-Raphson method is favored for its quadratic convergence, meaning the number of accurate bits roughly doubles with each iteration. We prove this by examining the relative error.

Let the relative error at iteration n be defined as:

$$\epsilon_n = 1 - D \cdot x_n \quad (8)$$

The error at the subsequent iteration, $n + 1$, is:

$$\epsilon_{n+1} = 1 - D \cdot x_{n+1} \quad (9)$$

Substituting the iteration formula for x_{n+1} :

$$\begin{aligned} \epsilon_{n+1} &= 1 - D(x_n(2 - D \cdot x_n)) \\ &= 1 - 2D \cdot x_n + D^2 \cdot x_n^2 \end{aligned}$$

Recognizing this as a perfect square binomial, we get:

$$\epsilon_{n+1} = (1 - D \cdot x_n)^2 \quad (10)$$

Therefore:

$$\epsilon_{n+1} = \epsilon_n^2 \quad (11)$$

This mathematically guarantees that if the initial error is less than 1 (e.g., 10^{-1}), the next error will be squared (10^{-2} , then 10^{-4}), demonstrating quadratic convergence.

2.3 The Mathematical Necessity of Normalization

Looking at the convergence relation $\epsilon_{n+1} = \epsilon_n^2$, the algorithm will only converge if the absolute value of the initial error is strictly less than 1. If $|\epsilon_0| \geq 1$, the error will square to a larger number, and the hardware will diverge to infinity.

Therefore, we must strictly satisfy:

$$\begin{aligned} |\epsilon_0| &< 1 \\ |1 - D \cdot x_0| &< 1 \\ -1 &< 1 - D \cdot x_0 < 1 \\ 0 &< x_0 < \frac{2}{D} \end{aligned}$$

If D is permitted to be any arbitrary value (e.g., $D = 10000$ or $D = 0.0001$), it is mathematically impossible to design a single, static hardware function $x_0(D)$ that guarantees x_0 falls within the microscopic window of $(0, 2/D)$ for all possible inputs.

The Normalization Solution: To guarantee convergence, we decouple the scale from the significant digits. Any binary number D can be expressed as:

$$D = d \cdot 2^k \quad (12)$$

where k is an integer representing the bit-shift, and d is the *normalized mantissa* strictly bounded within the interval $[0.5, 1.0)$.

By forcing the input into this fixed, narrow domain, we can easily derive a single initial guess function (such as $x_0 = 4(\sqrt{3} - 1) - 2d$) that mathematically guarantees $|\epsilon_0| < 1$ for all inputs. The Newton-Raphson iterations are then performed solely on the normalized value d to find $1/d$. Finally, the result is denormalized by shifting it by 2^{-k} to yield the true reciprocal $1/D$.

3 Making an Initial Guess

The convergence of the Newton-Raphson method depends on the relative error $\epsilon(d)$ of the initial guess $x_0(d)$, defined as:

$$\epsilon(d) = \frac{1/d - x_0(d)}{1/d} = 1 - d \cdot x_0(d) \quad (13)$$

To minimize this error over the normalized interval $d \in [0.5, 1]$, we apply the Chebyshev Equioscillation Theorem.

3.1 Unconstrained Linear Minimax Approximation

We first seek a general linear approximation $x_0(d) = a - bd$. The relative error function is:

$$\epsilon(d) = 1 - d(a - bd) = bd^2 - ad + 1 \quad (14)$$

This is a parabola with its vertex at $d_{vertex} = \frac{a}{2b}$. According to the **Chebyshev Equioscillation Theorem** (for minimax approximation), the optimal error function must alternate in sign with equal maximum magnitudes. Therefore, for this parabolic error curve, the errors at the boundaries ($d = 0.5$ and $d = 1.0$) must equal each other, and must be the exact negative of the error at the vertex $d_{vertex} = \frac{a}{2b}$.

First, equating the boundary errors:

$$\begin{aligned} \epsilon(0.5) &= \epsilon(1) \\ 0.25b - 0.5a + 1 &= b - a + 1 \implies a = \frac{3}{2}b \end{aligned}$$

Next, equating the upper boundary error to the negative of the vertex error:

$$\begin{aligned} \epsilon(1) &= -\epsilon\left(\frac{a}{2b}\right) \\ (b - a + 1) &= -\left[b\left(\frac{a}{2b}\right)^2 - a\left(\frac{a}{2b}\right) + 1\right] \end{aligned}$$

Substituting $a = 1.5b$ yields:

$$\begin{aligned} 1 - 0.5b &= -\left(1 - \frac{9b}{16}\right) \\ 2 &= \frac{17b}{16} \implies b = \frac{32}{17} \end{aligned}$$

Substituting b back into $a = 1.5b$ gives $a = \frac{48}{17}$, resulting in the optimal unconstrained linear guess:

$$x_0(d) = \frac{48}{17} - \frac{32}{17}d \quad (15)$$

3.2 Constrained Linear Approximation (Hardware Optimized)

The unconstrained result requires a DSP multiplier to compute $\frac{32}{17}d$. To avoid this, we constrain the slope to $b = 2$, allowing the multiplication to be performed via a simple bit-shift. The ansatz becomes $x_0(d) = a - 2d$, yielding the error function:

$$\epsilon(d) = 2d^2 - ad + 1 \quad (16)$$

The vertex is now at $d_{vertex} = \frac{a}{4}$. We equate the maximum boundary error at $d = 1$ to the negative of the vertex error:

$$\begin{aligned} \epsilon(1) &= -\epsilon\left(\frac{a}{4}\right) \\ 3 - a &= -\left[2\left(\frac{a}{4}\right)^2 - a\left(\frac{a}{4}\right) + 1\right] \\ 3 - a &= \frac{a^2}{8} - 1 \\ a^2 + 8a - 32 &= 0 \end{aligned}$$

Solving for a using the quadratic formula:

$$a = \frac{-8 \pm \sqrt{64 - 4(1)(-32)}}{2} = \frac{-8 \pm 8\sqrt{3}}{2} = 4(\sqrt{3} - 1) \quad (17)$$

The optimal multiplier-less initial guess is therefore:

$$x_0(d) = 4(\sqrt{3} - 1) - 2d \quad (18)$$

3.3 Quadratic Minimax Approximation

To further reduce initial error, a quadratic guess $x_0(d) = Ad^2 + Bd + C$ can be employed. The relative error becomes a cubic polynomial:

$$\epsilon(d) = -Ad^3 - Bd^2 - Cd + 1 \quad (19)$$

By Chebyshev's Equioscillation Theorem, the optimal error curve has four points of alternating maximal error in $[0.5, 1]$. Resolving the coefficients requires solving a non-linear system, typically done numerically via the Remez Algorithm. Rationalizing the numerical results for hardware yields:

$$x_0(d) = \frac{256}{99}d^2 - \frac{576}{99}d + \frac{420}{99} \quad (20)$$

This provides superior precision but mandates multiple hardware multipliers to compute d^2 and the subsequent scaling, but converges in lesser iterations compared to the linear guesses described before.

3.4 Comparison of Methods

Method	Formula/Mechanism	Precision (bits)	Hardware Cost
Linear Constrained	$x_0 = 4(\sqrt{3} - 1) - 2d$	≈ 3.5 bits	Very Low (1 shift, 1 sub)
Linear Unconstrained	$x_0 = \frac{48}{17} - \frac{32}{17}d$	≈ 4.0 bits	Low (1 mult, 1 sub)
Quadratic Minimax	Polynomial Interpolation	≈ 8.0 bits	High (Multipliers)

Table 1: Comparison of Initial Guess Methods for Newton-Raphson

4 Algorithm Summary

- Normalize denominator to the interval $[0.5, 1.0)$
- Compute smart initial guess (Linear or Quadratic)
- Perform Newton-Raphson iterations
- Adjust scaling (Denormalization)
- Multiply by numerator

5 Hardware Validation Test Cases

$$\frac{15}{23}, \frac{10}{79}, \frac{142}{51}, \frac{17}{64}, \frac{100}{127}, \frac{1}{255}$$

The testcases are carefully selected to comprehensively validate standard iteration logic, verify integer stability for improper fractions, evaluate the extreme upper and lower boundaries of the $[0.5, 1.0)$ approximation interval, and stress-test bit-level fractional precision.

6 Results

```
(venv) bhavan@bhavan-ubuntu:~/Academics/Sem-4/DSP Lab/Experiment-3$ python3 div.py
Test 15/23

After 1 iteration
Division by reciprocation using linear initial guess :
Ans = 0.6521670357968418 Original Ans = 0.6521739130434783 Error = 6.877246636505241e-06
Division by reciprocation using linear initial guess ( b = 2 ) :
Ans = 0.6521569227507961 Original Ans = 0.6521739130434783 Error = 1.699029268220631e-05
Division by reciprocation using quadratic initial guess :
Ans = 0.6521739129200613 Original Ans = 0.6521739130434783 Error = 1.2341694333173336e-10

After 2 iterations
Division by reciprocation using linear initial guess :
Ans = 0.652173912970957 Original Ans = 0.6521739130434783 Error = 7.25213222807497e-11
Division by reciprocation using linear initial guess ( b = 2 ) :
Ans = 0.6521739126008509 Original Ans = 0.6521739130434783 Error = 4.4262737919353867e-10
Division by reciprocation using quadratic initial guess :
Ans = 0.6521739130434783 Original Ans = 0.6521739130434783 Error = 0.0

After 3 iterations
Division by reciprocation using linear initial guess :
Ans = 0.6521739130434783 Original Ans = 0.6521739130434783 Error = 0.0
Division by reciprocation using linear initial guess ( b = 2 ) :
Ans = 0.6521739130434783 Original Ans = 0.6521739130434783 Error = 0.0
Division by reciprocation using quadratic initial guess :
Ans = 0.6521739130434783 Original Ans = 0.6521739130434783 Error = 0.0
```

Figure 1: Python example indicating that error decreases as we increase the number of iterations

```
(venv) bhavan@bhavan-ubuntu:~/Academics/Sem-4/DSP Lab/Experiment-3$ python3 div.py
Test 15/23 : After 2 iterations
Division by reciprocation using linear initial guess :
Ans = 0.652173912970957 Original Ans = 0.6521739130434783 Error = 7.25213222807497e-11
Division by reciprocation using linear initial guess ( b = 2 ) :
Ans = 0.6521739126008509 Original Ans = 0.6521739130434783 Error = 4.4262737919353867e-10
Division by reciprocation using quadratic initial guess :
Ans = 0.6521739130434783 Original Ans = 0.6521739130434783 Error = 0.0

Test 10/79 : After 2 iterations
Division by reciprocation using linear initial guess :
Ans = 0.12658227848098916 Original Ans = 0.12658227848101267 Error = 2.350897254643769e-14
Division by reciprocation using linear initial guess ( b = 2 ) :
Ans = 0.12658227847872414 Original Ans = 0.12658227848101267 Error = 2.288530476235451e-12
Division by reciprocation using quadratic initial guess :
Ans = 0.12658227848101264 Original Ans = 0.12658227848101267 Error = 2.7755575615628914e-17

Test 142/51 : After 2 iterations
Division by reciprocation using linear initial guess :
Ans = 2.7843137252674435 Original Ans = 2.784313725490196 Error = 2.227524831255323e-10
Division by reciprocation using linear initial guess ( b = 2 ) :
Ans = 2.784313724764062 Original Ans = 2.784313725490196 Error = 7.261338197395162e-10
Division by reciprocation using quadratic initial guess :
Ans = 2.784313725490196 Original Ans = 2.784313725490196 Error = 0.0

Test 17/64 : After 2 iterations
Division by reciprocation using linear initial guess :
Ans = 0.2656249999619217 Original Ans = 0.265625 Error = 3.8078318276291157e-11
Division by reciprocation using linear initial guess ( b = 2 ) :
Ans = 0.2656249999926736 Original Ans = 0.265625 Error = 7.326361739501408e-13
Division by reciprocation using quadratic initial guess :
Ans = 0.265625 Original Ans = 0.265625 Error = 0.0

Test 100/127 : After 2 iterations
Division by reciprocation using linear initial guess :
Ans = 0.7874015747636661 Original Ans = 0.7874015748031497 Error = 3.948352755855922e-11
Division by reciprocation using linear initial guess ( b = 2 ) :
Ans = 0.7874015745938008 Original Ans = 0.7874015748031497 Error = 2.0934887157153526e-10
Division by reciprocation using quadratic initial guess :
Ans = 0.7874015748031494 Original Ans = 0.7874015748031497 Error = 2.220446049250313e-16

Test 1/255 : After 2 iterations
Division by reciprocation using linear initial guess :
Ans = 0.003921568627114121 Original Ans = 0.00392156862745098 Error = 3.3685901290603226e-13
Division by reciprocation using linear initial guess ( b = 2 ) :
Ans = 0.003921568625732604 Original Ans = 0.00392156862745098 Error = 1.7183763093009397e-12
Division by reciprocation using quadratic initial guess :
Ans = 0.00392156862745098 Original Ans = 0.00392156862745098 Error = 0.0
```

Figure 2: Python output after 2 iterations

```

(venv) bhavan@bhavan-ubuntu:~/Academics/Sem-4/DSP Lab/Experiment-3$ iverilog -o div tb_div.v div.v
(venv) bhavan@bhavan-ubuntu:~/Academics/Sem-4/DSP Lab/Experiment-3$ vvp div
-----
COMBINATIONAL DIVISION TEST WITH ERROR ANALYSIS
-----
Test: 15 / 23
Expected : 0.65217391
Res Const: 0.69876714 | Error: 4.659322e-02
Res Lin  : 0.68933823 | Error: 3.716432e-02
Res Quad : 0.65459001 | Error: 2.416098e-03
-----
Test: 10 / 79
Expected : 0.12658228
Res Const: 0.13233033 | Error: 5.748050e-03
Res Lin  : 0.12982536 | Error: 3.243086e-03
Res Quad : 0.12785292 | Error: 1.270642e-03
-----
Test: 142 / 51
Expected : 2.78431373
Res Const: 2.96081810 | Error: 1.765044e-01
Res Lin  : 2.93658089 | Error: 1.522672e-01
Res Quad : 2.76921949 | Error: 1.509424e-02
-----
Test: 17 / 64
Expected : 0.26562500
Res Const: 0.25608949 | Error: 9.535510e-03
Res Lin  : 0.25000000 | Error: 1.562500e-02
Res Quad : 0.26294174 | Error: 2.683256e-03
-----
Test: 100 / 127
Expected : 0.78740157
Res Const: 0.73736580 | Error: 5.003577e-02
Res Lin  : 0.74678309 | Error: 4.061849e-02
Res Quad : 0.79320339 | Error: 5.801820e-03
-----
Test: 1 / 255
Expected : 0.00392157
Res Const: 0.00365631 | Error: 2.652595e-04
Res Lin  : 0.00370519 | Error: 2.163763e-04
Res Quad : 0.00395569 | Error: 3.411970e-05
-----
tb_div.v:30: $finish called at 60000 (1ps)
(venv) bhavan@bhavan-ubuntu:~/Academics/Sem-4/DSP Lab/Experiment-3$

```

Figure 3: Verilog - Using the initial guess (0 iterations)

```

(venv) bhavan@bhavan-ubuntu:~/Academics/Sem-4/DSP Lab/Experiment-3$ iverilog -o div tb_div.v div.v
(venv) bhavan@bhavan-ubuntu:~/Academics/Sem-4/DSP Lab/Experiment-3$ vvp div
-----
COMBINATIONAL DIVISION TEST WITH ERROR ANALYSIS
-----
Test: 15 / 23
Expected : 0.65217391
Res Const: 0.64884515 | Error: 3.328758e-03
Res Lin  : 0.65005609 | Error: 2.117819e-03
Res Quad : 0.65216496 | Error: 8.950901e-06
-----
Test: 10 / 79
Expected : 0.12658228
Res Const: 0.12632126 | Error: 2.610186e-04
Res Lin  : 0.12649919 | Error: 8.309128e-05
Res Quad : 0.12656952 | Error: 1.275780e-05
-----
Test: 142 / 51
Expected : 2.78431373
Res Const: 2.77312468 | Error: 1.118904e-02
Res Lin  : 2.77598662 | Error: 8.327110e-03
Res Quad : 2.78423190 | Error: 8.182432e-05
-----
Test: 17 / 64
Expected : 0.26562500
Res Const: 0.26528269 | Error: 3.423095e-04
Res Lin  : 0.26470588 | Error: 9.191185e-04
Res Quad : 0.26559789 | Error: 2.710521e-05
-----
Test: 100 / 127
Expected : 0.78740157
Res Const: 0.78422203 | Error: 3.179546e-03
Res Lin  : 0.78530625 | Error: 2.095326e-03
Res Quad : 0.78735882 | Error: 4.275064e-05
-----
Test: 1 / 255
Expected : 0.00392157
Res Const: 0.00390362 | Error: 1.794496e-05
Res Lin  : 0.00390963 | Error: 1.193979e-05
Res Quad : 0.00392127 | Error: 2.982570e-07
-----
tb_div.v:30: $finish called at 60000 (1ps)
(venv) bhavan@bhavan-ubuntu:~/Academics/Sem-4/DSP Lab/Experiment-3$

```

Figure 4: Verilog - After 1 iteration

```

(venv) bhavan@bhavan-ubuntu:~/Academics/Sem-4/DSP Lab/Experiment-3$ iverilog -o div tb_div.v div.v
(venv) bhavan@bhavan-ubuntu:~/Academics/Sem-4/DSP Lab/Experiment-3$ vvp div
-----
COMBINATIONAL DIVISION TEST WITH ERROR ANALYSIS
-----
Test: 15 / 23
Expected : 0.65217391
Res Const: 0.65215692 | Error: 1.699008e-05
Res Lin  : 0.65216703 | Error: 6.879639e-06
Res Quad : 0.65217391 | Error: 2.753475e-09
-----
Test: 10 / 79
Expected : 0.12658228
Res Const: 0.12658174 | Error: 5.388467e-07
Res Lin  : 0.12658222 | Error: 5.455900e-08
Res Quad : 0.12658228 | Error: 2.404934e-09
-----
Test: 142 / 51
Expected : 2.78431373
Res Const: 2.78426876 | Error: 4.496630e-05
Res Lin  : 2.78428882 | Error: 2.490561e-05
Res Quad : 2.78431372 | Error: 2.045257e-09
-----
Test: 17 / 64
Expected : 0.26562500
Res Const: 0.26562456 | Error: 4.433095e-07
Res Lin  : 0.26562182 | Error: 3.181398e-06
Res Quad : 0.26562500 | Error: 3.725290e-09
-----
Test: 100 / 127
Expected : 0.78740157
Res Const: 0.78738873 | Error: 1.284401e-05
Res Lin  : 0.78739600 | Error: 5.579693e-06
Res Quad : 0.78740157 | Error: 2.933299e-09
-----
Test: 1 / 255
Expected : 0.00392157
Res Const: 0.00392149 | Error: 8.219013e-08
Res Lin  : 0.00392153 | Error: 3.748665e-08
Res Quad : 0.00392157 | Error: 2.337437e-10
-----
tb_div.v:30: $finish called at 60000 (1ps)
(venv) bhavan@bhavan-ubuntu:~/Academics/Sem-4/DSP Lab/Experiment-3$

```

Figure 5: Verilog - After 2 iterations

```

(venv) bhavan@bhavan-ubuntu:~/Academics/Sem-4/DSP Lab/Experiment-3$ iverilog -o div tb_div.v div.v
(venv) bhavan@bhavan-ubuntu:~/Academics/Sem-4/DSP Lab/Experiment-3$ vvp div
-----
COMBINATIONAL DIVISION TEST WITH ERROR ANALYSIS
-----
Test: 15 / 23
Expected : 0.65217391
Res Const: 0.65217391 | Error: 2.753475e-09
Res Lin  : 0.65217391 | Error: 2.753475e-09
Res Quad : 0.65217391 | Error: 9.718149e-10
-----
Test: 10 / 79
Expected : 0.12658228
Res Const: 0.12658228 | Error: 2.404934e-09
Res Lin  : 0.12658228 | Error: 2.404934e-09
Res Quad : 0.12658228 | Error: 2.404934e-09
-----
Test: 142 / 51
Expected : 2.78431373
Res Const: 2.78431373 | Error: 5.405323e-09
Res Lin  : 2.78431373 | Error: 5.405323e-09
Res Quad : 2.78431373 | Error: 5.405323e-09
-----
Test: 17 / 64
Expected : 0.26562500
Res Const: 0.26562500 | Error: 3.725290e-09
Res Lin  : 0.26562500 | Error: 0.000000e+00
Res Quad : 0.26562500 | Error: 3.725290e-09
-----
Test: 100 / 127
Expected : 0.78740157
Res Const: 0.78740157 | Error: 2.933299e-09
Res Lin  : 0.78740157 | Error: 2.933299e-09
Res Quad : 0.78740157 | Error: 2.933299e-09
-----
Test: 1 / 255
Expected : 0.00392157
Res Const: 0.00392157 | Error: 2.337437e-10
Res Lin  : 0.00392157 | Error: 2.337437e-10
Res Quad : 0.00392157 | Error: 2.337437e-10
-----
tb_div.v:30: $finish called at 60000 (1ps)
(venv) bhavan@bhavan-ubuntu:~/Academics/Sem-4/DSP Lab/Experiment-3$

```

Figure 6: Verilog - After 3 iterations

7 Observations

- The Verilog simulation results show rapid convergence across successive Newton–Raphson iterations, closely matching the theoretical behavior predicted by the Python reference model.
- During the early stages (Iteration 0 and Iteration 1), the quadratic and linear initial guesses exhibit significantly lower initial errors compared to the constrained guess, as expected.
- By Iteration 3, the absolute error for all three approximation methods converges to a similar lower bound, typically in the range of 10^{-9} to 10^{-10} .
- This error plateau in the Verilog results is caused by the finite precision of the 32-bit Q4.28 fixed-point representation.
- The smallest representable fractional value in Q(4,28) format is:

$$2^{-28} \approx 3.72 \times 10^{-9}$$

which defines the theoretical precision limit of the hardware implementation.

- In contrast, the Python reference model uses 64-bit double-precision floating-point arithmetic, allowing convergence toward machine epsilon (approximately 10^{-16}) within the same number of iterations.
- The quadratic initial guess requires more hardware but converges in less iterations than the linear guess. This is a trade-off that exists here.

8 Conclusion

The Newton-Raphson method provides fast quadratic convergence for hardware division via reciprocal computation. Normalizing the denominator to the $[0.5, 1.0)$ interval and applying minimax-optimized initial approximations significantly reduces the required iteration count. Furthermore, the Q(4,28) fixed-point format ensures high numerical precision, achieving absolute errors on the order of 10^{-9} , while maintaining a fast and resource-efficient hardware datapath.